

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

*I **A Mohammad Naheed**(Reg No: **192525336**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A Mohammad Naheed

1. Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'

Query Processing and Query Optimization in DBMS

Query processing is the process of converting an SQL query written by a user into an efficient sequence of operations that can be executed by the DBMS to produce the required result. The main objective is to obtain the result correctly while using minimum time, CPU, memory, and disk I/O.

Steps in Query Processing

1. Parsing and Validation:

The DBMS first checks the SQL query for syntax and semantic correctness. It verifies whether the tables, columns, operators, and functions used in the query are valid.

2. Translation:

The SQL query is converted into an internal representation, usually a **relational algebra expression** or query tree.

3. Query Optimization:

The DBMS generates different possible execution plans for the query. It compares their estimated costs and selects the most efficient plan.

4. Query Execution:

The selected execution plan is passed to the query execution engine. It performs operations such as table scanning, filtering, sorting, and joining.

5. Result Generation:

Finally, the processed data is returned to the user or application.

How Does the Query Optimizer Choose the Best Plan?

The optimizer considers:

- Number of records in tables
- Available indexes
- Selectivity of conditions
- Join order
- Join algorithms
- Disk I/O cost
- CPU and memory requirements
- Database statistics For

example:

```
SELECT * FROM Student WHERE  
Student_ID = 101;
```

If Student_ID has an index, the optimizer may use an **index search** instead of scanning every record.

Conclusion

Query optimization is important because the same SQL query can have many execution plans with different performance. The optimizer selects the plan with the **lowest estimated execution cost**, making database operations faster and more efficient.

1. Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'

Heuristic Optimization vs Cost-Based Optimization

Query optimization is the process of selecting an efficient execution plan for a query. Two major approaches are **heuristic optimization** and **cost-based optimization**.

Heuristic Optimization

Heuristic optimization uses predefined rules or techniques to improve a query without calculating the exact cost of every possible execution plan.

Common heuristics include:

1. Perform **selection operations early**.
2. Perform **projection operations early**.
3. Reduce the number of records before performing joins.
4. Choose efficient join operations.
5. Avoid unnecessary operations.

For example, instead of joining two complete tables and then applying a condition, the DBMS can first filter the required records and then perform the join.

Cost-Based Optimization

Cost-based optimization generates multiple possible execution plans and estimates the cost of each plan. The optimizer selects the plan with the lowest estimated cost.

It considers:

- Disk I/O
- CPU usage
- Memory usage
- Number of records
- Table size
- Index availability
- Join methods
- Selectivity of conditions

For example, for joining two tables, the optimizer may compare:

- Nested-loop join
- Hash join
- Merge join

and select the one with the lowest estimated cost.

Difference

Heuristic Optimization	Cost-Based Optimization
Uses predefined rules	Uses estimated execution cost
Faster optimization	More optimization computation
Does not require detailed statistics	Requires database statistics
May not always produce the best plan	Usually produces a better plan
Suitable for simple queries	Suitable for complex queries

When to Use?

Heuristic optimization is useful when quick optimization is required or statistics are unavailable. **Cost-based optimization** is preferred for complex queries and large databases where performance is very important.

Conclusion

Modern DBMSs commonly use **cost-based optimization**, often combined with heuristic rules, to achieve efficient query execution.

2. Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'

Database Connectivity and JDBC vs ODBC

Database connectivity allows an application to communicate with a database to perform operations such as inserting, updating, deleting, and retrieving data.

For example, an online banking application written in Java needs to connect to a database to retrieve customer account information.

Steps for Database Connectivity Using JDBC

1. Load the JDBC Driver:

The appropriate database driver is loaded.

2. Establish Connection:

The Java application creates a connection with the database using a connection URL, username, and password.

3. Create Statement:

A Statement or PreparedStatement is created to execute SQL commands.

4. Execute Query:

SQL commands such as SELECT, INSERT, UPDATE, and DELETE are executed.

5. Process Results:

For a SELECT query, the returned data is processed using a ResultSet.

6. Close Resources:

The connection, statement, and result set are closed after completing the operation.

Example:

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/studentdb", "root",
    "password"); PreparedStatement ps
=
    con.prepareStatement("SELECT * FROM Student"); ResultSet rs =
ps.executeQuery();
```

JDBC

JDBC (Java Database Connectivity) is a Java API used to connect Java applications with databases.

ODBC

ODBC (Open Database Connectivity) is a standard API that allows applications to communicate with different database systems.

JDBC vs ODBC

JDBC	ODBC
Designed mainly for Java	Language-independent standard
Java-based API	Uses ODBC drivers
Platform independent with Java	Depends more on native drivers
Commonly used in Java applications	Used by many programming environments
Provides direct Java integration	General database connectivity

Conclusion

JDBC is the preferred choice for Java applications because it provides direct and convenient communication between Java programs and databases.

3. Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'

ACID Properties with Real-World Examples

ACID represents **A**tomicity, **C**onsistency, **I**solation, and **D**urability. These properties ensure that database transactions are reliable and maintain the correctness of data.

A **transaction** is a logical unit of work consisting of one or more database operations.

1. Atomicity

Atomicity means a transaction is treated as a single unit. Either **all operations are completed or none of them are applied**.

Example:

Suppose ₹5,000 is transferred from Account A to Account B.

1. ₹5,000 is deducted from A.
2. ₹5,000 is added to B.

If the system crashes after deduction but before addition, the entire transaction must be rolled back.

Therefore, atomicity prevents partial transactions.

2. Consistency

Consistency ensures that a transaction changes the database from **one valid state to another valid state**.

For example, if a bank account cannot have a negative balance, a transaction must not violate this constraint.

3. Isolation

Isolation ensures that multiple transactions running simultaneously do not interfere with each other.

Example:

Suppose only one cinema ticket is available. Two customers attempt to purchase it simultaneously. Isolation ensures that both transactions do not successfully purchase the same seat.

4. Durability

Durability means that once a transaction is successfully committed, its changes are permanently stored even if the system crashes.

Example:

After an online payment shows "**Payment Successful**", the transaction record should remain available even if the server loses power.

Importance of ACID

ACID properties are particularly important in:

- Banking
- Online payments
- Railway and airline reservations
- E-commerce
- Hospital systems
- Financial applications

Conclusion

ACID properties ensure that transactions are **reliable, correct, recoverable, and consistent**, making them essential for critical database applications.

5. Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'

Concurrency Problems and Two-Phase Locking

Concurrency control is the process of managing multiple transactions executing simultaneously while maintaining database consistency.

Without proper concurrency control, several problems can occur.

1. Dirty Read

A dirty read occurs when one transaction reads data that has been modified by another transaction but **has not yet been committed**.

Example:

- T1 changes balance from ₹10,000 to ₹5,000.
- T2 reads ₹5,000.
- T1 rolls back.

- Actual balance becomes ₹10,000.

T2 has read incorrect temporary data.

2. Lost Update

A lost update occurs when two transactions update the same data and one update overwrites another.

Example:

- T1 reads balance = ₹10,000.
- T2 reads balance = ₹10,000.
- T1 changes it to ₹12,000.
- T2 changes it to ₹9,000.

The update made by T1 is lost.

3. Unrepeatable Read

An unrepeatable read occurs when a transaction reads the same data twice but gets different values because another transaction modified it between the two reads.

Example:

- T1 reads salary = ₹30,000.
- T2 updates salary to ₹35,000 and commits.
- T1 reads the salary again and gets ₹35,000.

Two-Phase Locking (2PL)

2PL controls concurrency using locks. It has two

phases:

Growing Phase

The transaction can **acquire locks** but cannot release them.

Shrinking Phase

The transaction can **release locks** but cannot acquire new locks. Locks can be:

- **Shared Lock (S):** Used for reading.
- **Exclusive Lock (X):** Used for writing.

How 2PL Helps

By controlling when transactions can read or modify data, 2PL prevents conflicting operations and guarantees **conflict serializability**.

Conclusion

Concurrency problems can cause incorrect results and data inconsistency. **Two-Phase Locking provides controlled access to shared data and ensures transactions execute in a safe and serializable manner.**

6. Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'

Deadlock in DBMS

A **deadlock** occurs when two or more transactions wait indefinitely for resources held by each other.

Example

Suppose:

- T1 holds Lock A and waits for Lock B.
- T2 holds Lock B and waits for Lock A. Neither

transaction can continue.

This creates a deadlock.

Deadlock Detection

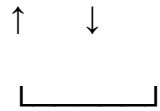
A common technique is the **Wait-For Graph**. In this graph:

- Each transaction is represented as a node.
- An edge $T1 \rightarrow T2$ means T1 is waiting for a resource held by T2.

If the graph contains a **cycle**, a deadlock exists.

Example:

$T1 \rightarrow T2$



This cycle indicates a deadlock.

Deadlock Handling

Once a deadlock is detected, the DBMS can select one transaction as a **victim**, roll it back, release its locks, and allow the remaining transactions to continue.

Deadlock Prevention Strategies

1. Wait-Die

This method uses transaction timestamps.

- Older transaction can wait for a younger transaction.
- Younger transaction requesting a resource held by an older transaction is rolled back.

2. Wound-Wait

- Older transaction can force a younger transaction to roll back.
- Younger transaction waits if the resource is held by an older transaction.

3. Resource Ordering

All transactions request resources in a fixed order.

For example, if resources must always be requested in the order:

$A \rightarrow B \rightarrow C$

then circular waiting can be avoided.

Conclusion

Deadlocks can stop transactions from progressing. DBMSs use **detection, prevention, and recovery techniques** to handle them efficiently.

7.Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'

Immediate Update vs Deferred Update Recovery

Database recovery is the process of restoring the database to a consistent state after failures such as system crashes, transaction failures, or power failures.

Two important recovery techniques are **Immediate Update** and **Deferred Update**. **Immediate Update**

In immediate update, changes made by a transaction can be written to the database **before the transaction commits**.

Therefore, if a failure occurs, the DBMS may need:

- **UNDO** – reverse changes made by uncommitted transactions.
- **REDO** – repeat changes of committed transactions if necessary.

Example

Suppose T1 changes account balance from ₹10,000 to ₹8,000. The change may be written to disk before T1 commits. If T1 fails, the system must undo the change.

Deferred Update

In deferred update, changes are not written to the actual database until the transaction **commits**.

If a transaction fails before committing, its changes can simply be discarded.

Recovery generally requires **REDO** for committed transactions.

Comparison

Immediate Update	Deferred Update
Changes may be written before commit	Changes written after commit

Requires UNDO and possibly REDO	Mainly requires REDO
More flexible	Simpler recovery
Can be more complex	Easier to implement
Suitable for high-performance systems	Suitable where simple recovery is preferred

Scenario

Immediate update is useful when the system needs changes to be written continuously and efficiently. Deferred update is useful when **simple recovery and reduced rollback complexity** are preferred.

Conclusion

Both methods ensure database recovery after failures. Immediate update provides flexibility but requires more complex recovery, while deferred update provides simpler recovery.

8. Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'

Shadow Paging

Shadow paging is a database recovery technique that maintains two page tables:

1. Shadow Page Table

2. Current Page Table

The shadow page table points to the original database pages and remains unchanged during a transaction.

When a transaction modifies a page, the DBMS creates a **new copy** of that page instead of modifying the original page.

Working

Initially:

Shadow Page Table



Original Pages

During the transaction:

Shadow Page Table → Original Pages

Current Page Table → Modified/New Pages

If the transaction commits, the current page table becomes the new valid page table.

If the transaction fails, the system simply uses the shadow page table to return to the previous state.

Advantages

1. Simple recovery mechanism.
2. No traditional UNDO operation is required.
3. Fast rollback.
4. Original pages remain unchanged until commit.
5. Easy recovery after system failure.

Limitations

1. Requires additional storage.
2. Can cause fragmentation.
3. Page-table management can be expensive.
4. Difficult to handle very large databases efficiently.
5. Concurrent transactions can make implementation complicated.

Shadow Paging vs Log-Based Recovery

Shadow Paging	Log-Based Recovery
Uses page tables	Uses transaction logs
Simple rollback	Supports UNDO/REDO
No traditional logging required	Requires log records
Can cause fragmentation	More scalable
Page copying overhead	Logging overhead

Conclusion

Shadow paging is useful because it provides a simple recovery mechanism, but **log-based recovery is generally preferred for large modern database systems** because it offers greater flexibility and scalability.

9. Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'

Serializability and Precedence Graph

Serializability is an important concept in concurrency control. A schedule is serializable if its result is equivalent to the result of executing the transactions **one after another in some serial order**.

Serial Schedule

Consider:

$T1 \rightarrow T2$

T1 completes first, followed by T2. This is a serial schedule.

Concurrent Schedule

Transactions may execute their operations in an interleaved manner: T1: Read(A)

T2: Read(B) T1:

Write(A) T2:

Write(B)

If the result is equivalent to some serial execution, the schedule is serializable.

Conflict Serializability

A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

Precedence Graph

A precedence graph is used to test conflict serializability.

Steps

1. Create a node for every transaction.

2. Identify conflicting operations.

Two operations conflict when:

- They belong to different transactions.
- They access the same data item.
- At least one operation is a write.

Conflicts include:

- Read-Write
- Write-Read
- Write-Write

3. Add an edge.

If T1 performs the conflicting operation before T2:

$T1 \rightarrow T2$

4. Check for cycles.

- No cycle $\rightarrow$ Conflict serializable
- Cycle $\rightarrow$ Not conflict serializable

Example

Suppose:

$T1 \rightarrow T2 \ T2$

$\rightarrow T3 \ T1 \rightarrow$

$T3$

There is no cycle, so the schedule is conflict serializable. If we have:

$T1 \rightarrow T2 \ T2$

$\rightarrow T1$

there is a cycle, so the schedule is **not conflict serializable**. **Conclusion**

Precedence graphs provide a simple and effective method for checking whether a concurrent schedule maintains the same correctness as a serial schedule.

10. Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'

Basic 2PL, Strict 2PL and Rigorous 2PL

Two-Phase Locking (2PL) is a concurrency control protocol that ensures conflict serializability by controlling the acquisition and release of locks.

It has two phases:

1. Growing Phase

The transaction can acquire locks but cannot release them.

2. Shrinking Phase

The transaction can release locks but cannot acquire new locks. There are three important variations.

Basic 2PL

In Basic 2PL:

- Locks are acquired during the growing phase.
- Locks are released during the shrinking phase.
- Once the first lock is released, no new lock can be acquired.

It guarantees **conflict serializability** but can allow **cascading rollbacks**.

Strict 2PL

In Strict 2PL:

- All **exclusive/write locks** are held until the transaction commits or aborts.
- Other transactions cannot read or modify data written by an uncommitted transaction.

Advantages:

- Prevents dirty reads of uncommitted writes.
- Prevents cascading rollback.
- Simplifies recovery.
- Ensures strict schedules.

Rigorous 2PL

In Rigorous 2PL:

- Both **shared/read locks and exclusive/write locks** are held until commit or abort.

It provides stronger locking guarantees than Strict 2PL.

Comparison

Feature	Basic 2PL	Strict 2PL	Rigorous 2PL
Conflict serializable	Yes	Yes	Yes
X-lock held until commit	Not necessarily	Yes	Yes
S-lock held until commit	Not necessarily	Not necessarily	Yes
Cascading rollback	Possible	Prevented	Prevented
Concurrency	Higher	Good	Lower
Recovery	More complex	Easier	Easier

Which Is Commonly Used?

Strict 2PL is commonly used because it provides a good balance between **concurrency, consistency, and recoverability**. It prevents transactions from reading uncommitted changes and makes recovery easier without holding all shared locks until commit.

Conclusion

Basic 2PL provides serializability, Strict 2PL additionally provides strong recoverability, and Rigorous 2PL provides the strongest locking discipline. In practical DBMS environments, **Strict 2PL is widely preferred** because of its balance between safety and performance.